Advanced Software Engineering VU [194.187]

Maria Christakis

TU Wien

https://mariachris.github.io

Abstract interpretation: Exercises

Exercise 1: Abstract Interpretation - Parity Domain

Consider the parity domain which has abstract elements $\{\bot, Even, Odd, \top\}$ where $Even$ is the set of all even integers including 0 and Odd is the set of all odd integers.

First define a join operator for the given domain. Then perform Abstract Interpretation on f. To this end construct the (abstract) control flow graph, where the nodes are the abstract states and the edges are the statements of f. Finally establish that the asserted condition holds.

```
0  void f(int n){
1      int p = 13;
2      int m = 101;
3      if (n % 2 == 0){
4          if(m > 0){
5              p = p + n;
6              m = m - 1;
7              goto 4;
8          }
9      }
10     p = p - 1;
11     assert p % 2 == 0;
12 }
```

Exercise 2: Abstract Interpretation - Comparison of Domains

Give either a program that can be verified with an abstract interpreter using the parity domain but not with the interval domain or show that such a program cannot exist.
Now give either a program that can be verified using the sign domain but not with the interval domain or show that such a program cannot exist

Exercise 2: Abstract Interpretation - Comparison of Domains

Give either a program that can be verified with an abstract interpreter using the parity domain but not with the interval domain or show that such a program cannot exist.
Now give either a program that can be verified using the sign domain but not with the interval domain or show that such a program cannot exist

- The program from Exercise 1 can be verified with the parity domain but not with the interval domain
  - Any other program?

Exercise 2: Abstract Interpretation - Comparison of Domains

Give either a program that can be verified with an abstract interpreter using the parity domain but not with the interval domain or show that such a program cannot exist.
Now give either a program that can be verified using the sign domain but not with the interval domain or show that such a program cannot exist

- The program from Exercise 1 can be verified with the parity domain but not with the interval domain
  - Any other program?

- The interval domain is more precise than the sign domain
  - We can map all elements of the sign domain to elements of the interval domain
  - So, there cannot exist a program that is verified with the sign domain but not with the interval domain

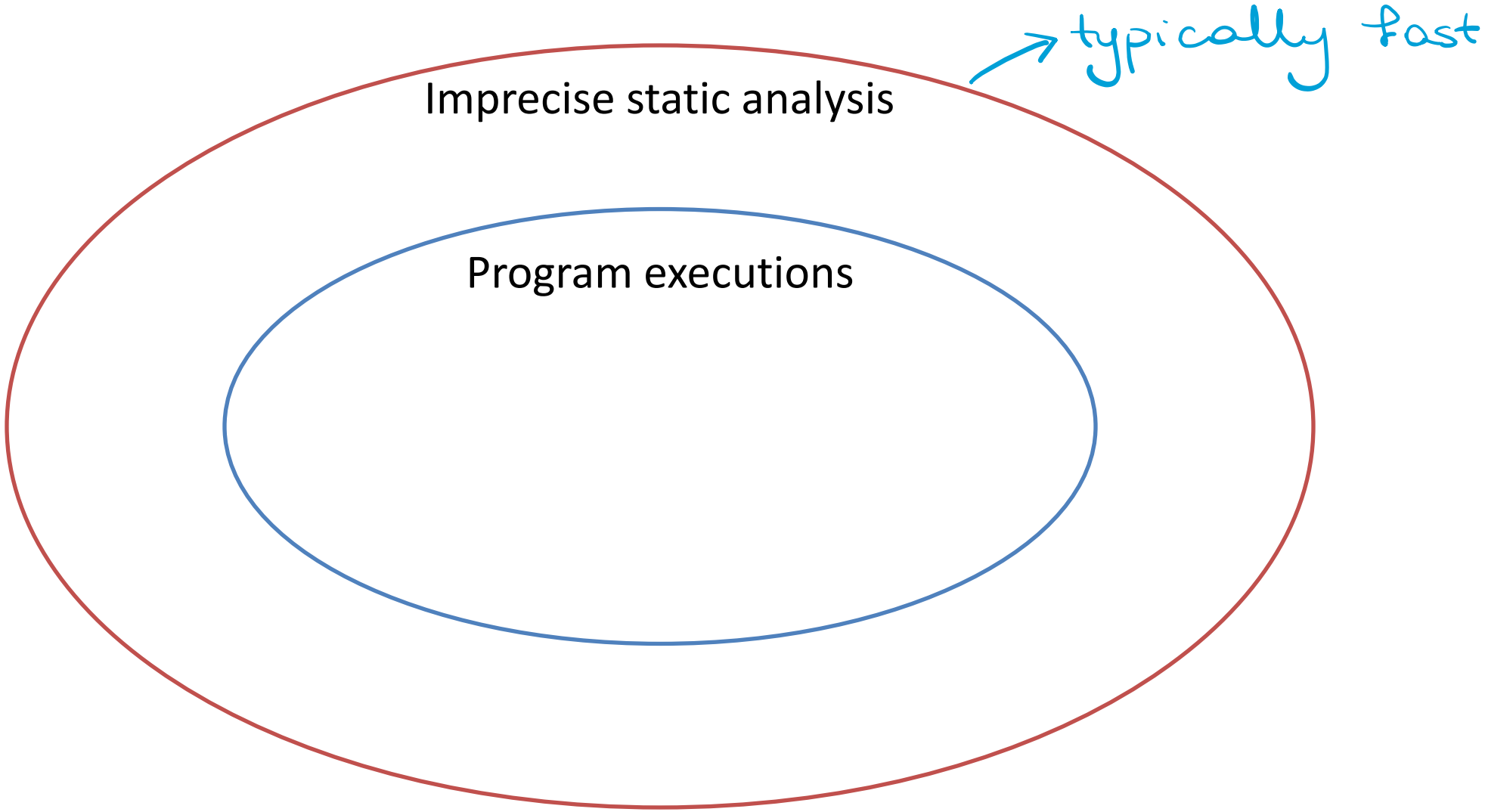
Exercise 3: Abstract interpretation – Intervals

Let's analyze the same program as before using the interval domain and widening

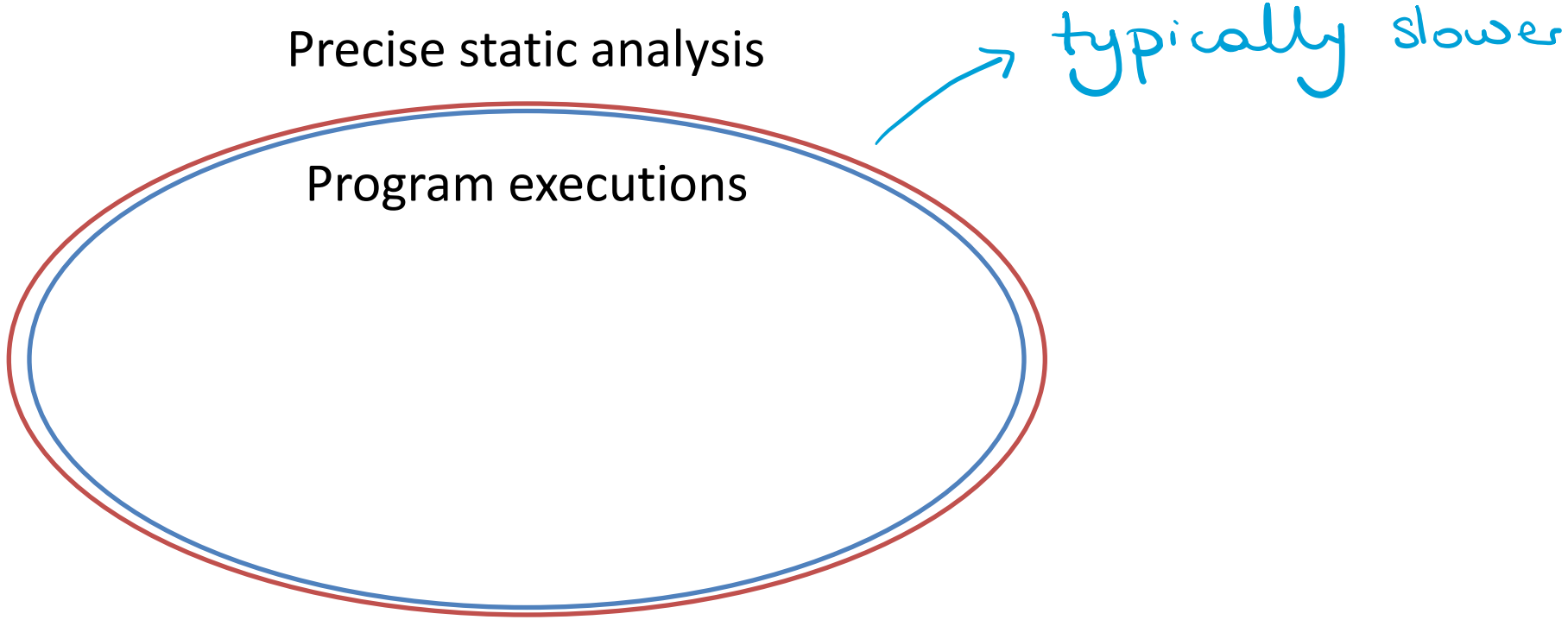
```
0  void f(int n){
1      int p = 13;
2      int m = 101;
3      if (n % 2 == 0){
4          if(m > 0){
5              p = p + n;
6              m = m - 1;
7              goto 4;
8          }
9      }
10     p = p - 1;
11     assert p % 2 == 0;
12 }
```

Unsound static analysis: Theory

Imprecision in static analysis



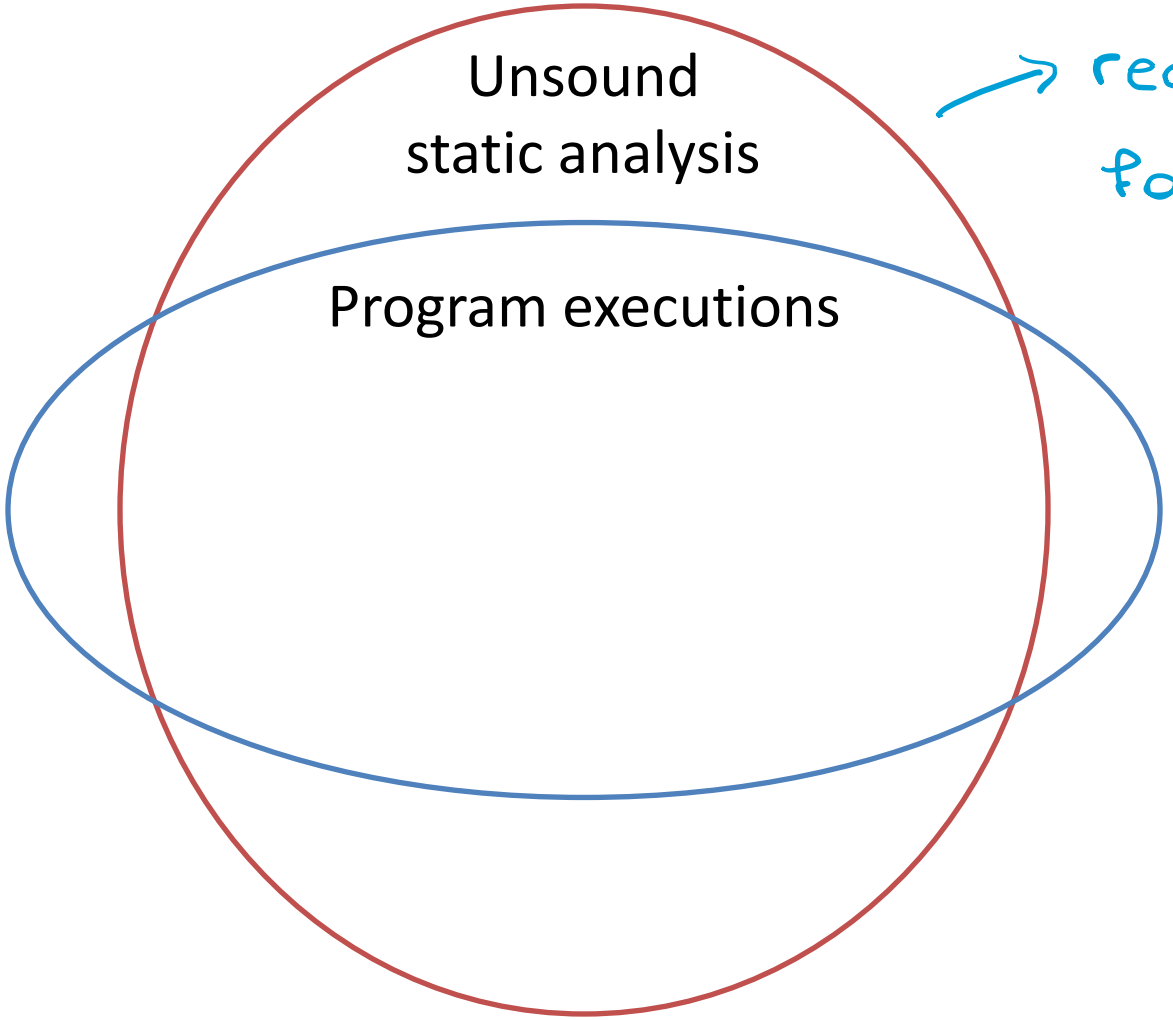
Precision in static analysis



Compromises in static analysis

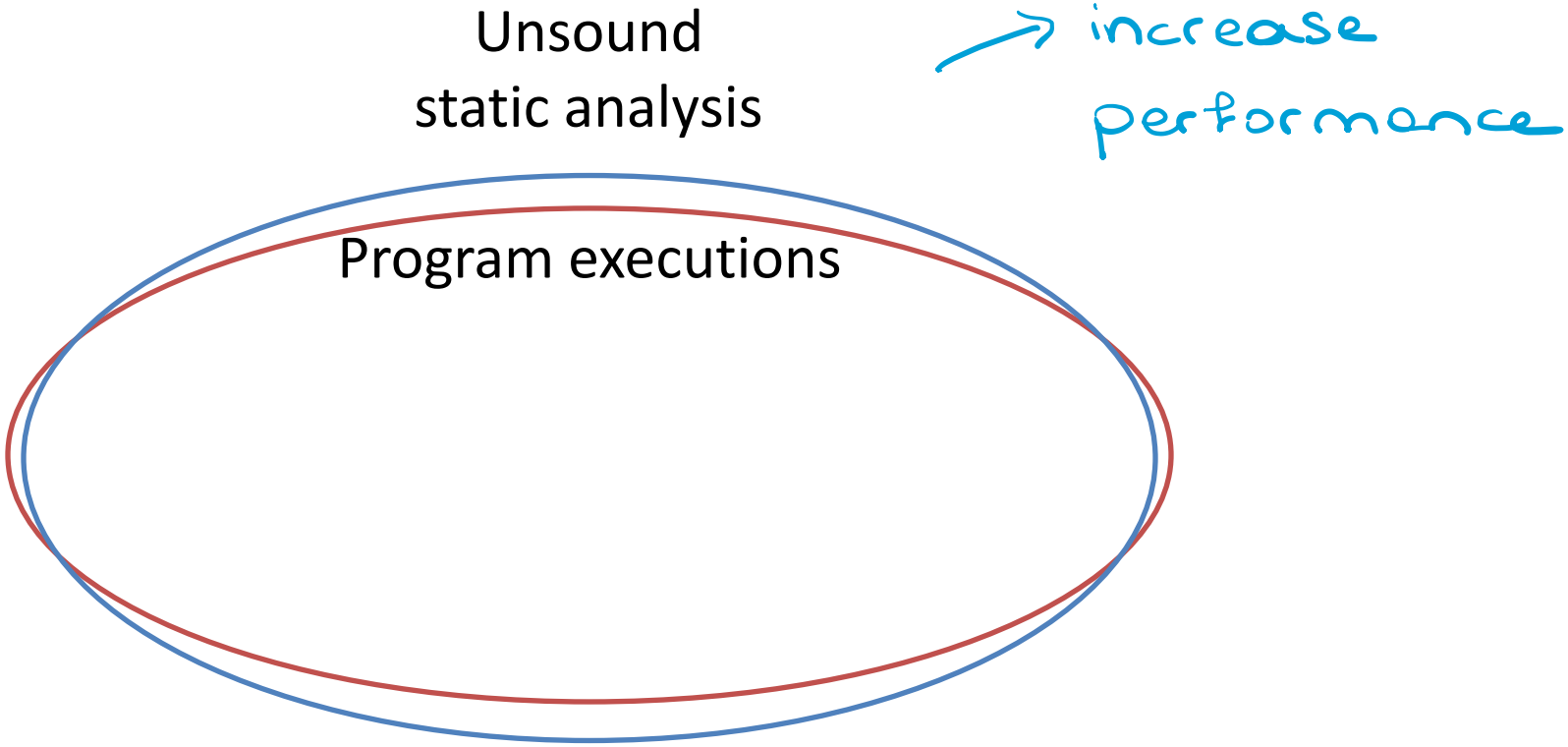
- Static analyzers may choose to be unsound
  - To reduce the number of false positives
  - To increase performance
  - …

Compromises in static analysis: Example



→ reduce number of false positives

Compromises in static analysis: Example



Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?

Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?
  - Yes! A sound analysis can be used for verification

Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?
  - Yes! A sound analysis can be used for verification

- Can we be sure about a proof generated with an unsound analysis?

Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?
  - Yes! A sound analysis can be used for verification

- Can we be sure about a proof generated with an unsound analysis?
  - No! An unsound analysis can be used for bug finding

Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?
  - Yes! A sound analysis can be used for verification
- Can we be sure about a proof generated with an unsound analysis?
  - No! An unsound analysis can be used for bug finding
- Can we be sure about a proof generated with an imprecise analysis?

Proofs with static analysis

- Can we be sure about a proof generated with a sound analysis?
  - Yes! A sound analysis can be used for verification
- Can we be sure about a proof generated with an unsound analysis?
  - No! An unsound analysis can be used for bug finding
- Can we be sure about a proof generated with an imprecise analysis?
  - Is it sound?

Flagship unsound static analyses: Application

Extended static checking for Java

- Called ESC/Java
- Performs modular checking
  - It operates on one piece of a program at a time – it is not necessary to have the source of the whole program in order to run the analysis
  - It considers future code as well
- The cost of modular checking is that annotations are needed

Example: Modularity and annotations

```
class Bag {
  int size;
  int[] elements;









  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  int[] elements;









  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

Possible null dereference

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  int[] elements;




  //@ requires input != null;
  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

ESC/Java assumes the precondition on method entry and checks it at call sites

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  int[] elements;




  //@ requires input != null;
  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

Possible null dereference

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  /*@non_null*/ int[] elements;




  //@ requires input != null;
  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  /*@non_null*/ int[] elements;




  //@ requires input != null;
  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

Possible index out of bounds

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

Example: Modularity and annotations

```
class Bag {
  int size;
  /*@non_null*/ int[] elements;

  //@invariant 0 <= size &&
    size <= elements.length;

  //@ requires input != null;
  Bag(int[] input) {
    size = input.length;
    elements = new int[size];
    System.arraycopy(input, 0,
      elements, 0, size);
  }
```

```
  int extractMin() {
    int min = Integer.MAX_VALUE;
    int minIndex = 0;
    for (int i = 0; i < size; i++) {
      if (elements[i] < min) {
        min = elements[i];
        minIndex = i;
      }
    }
    if (size > 0} {
      size--;
      elements[minIndex] = elements[size];
    }
    return min;
  }
}
```

An invariant is meant to hold at every method boundary for every initialized instance of a class

Unsoundness in ESC/Java

- ESC/Java compromises soundness to
  - Increase performance
  - Reduce the annotation overhead
- Examples of its unsoundness are
  - Loop unrolling
  - Partial checking of invariants
  - …

Levels of unsoundness

- Sound analyses for a small set of (perhaps restricted) languages
- Soundy analyses for most other languages
  - See "In defense of soundiness: A manifesto"
  - E.g., pointer arithmetic in C or reflection in Java are always ignored
- Unsound analyses for practical purposes
  - E.g., performance, precision, annotation overhead, modularity, etc.

The good news: Part 1

When the sources of unsoundness are well designed and thought through,

the static analysis might not miss any errors most of the time

See “An experimental evaluation of deliberate unsoundness in a static program analyzer”

The good news: Part 2

Unsound static analyzers may become sound if they document their unsoundness

See “Collaborative verification and testing with explicit assumptions”